Cover

GORM

ORM in Go for Beginers

Anuchit Chalothorn

Preface

Most Go developers meet SQL long before they meet an ORM, and the first ORM they meet tends to be the one that arrives with a tutorial. This book is for the moment after that: you have written a few queries by hand, you have started to repeat yourself, and you want to know whether GORM is worth putting between your structs and your database — and exactly what it will cost you if you do.

Who this book is for

This book is written for Go developers who are new to GORM, or new to ORMs in general. You do not need prior GORM experience. You do need to be comfortable with Go itself: structs, interfaces, error handling, and running a program from the command line. If you can read a function signature and reason about what it returns, you have enough to start.

You will get the most out of this book if you have a real project in mind — a service that stores users, orders, or events — because the chapters build toward the decisions that project will force on you. If you have never run a database at all, Chapter 2 walks through installing a driver and opening a connection from nothing.

This book is not a reference manual. GORM's full API surface is large, and the official documentation at https://gorm.io/docs/ is the right place to look up an individual method. What this book adds is the order: which concept you need before which other one, where the sharp edges are, and why the API behaves the way it does.

What you will learn

The book starts with the two things GORM is built on: declaring models, and the conventions that map your Go structs onto tables and columns. Once that foundation is in place, the four core operations — create, query, update, and delete — each get a chapter, because most bugs in day-to-day GORM code live in the details of one of them.

From there the book moves into the parts that separate working code from code you can maintain. You will model relationships between tables and learn how to load related data without falling into the N+1 trap. You will use migrations to evolve a schema without losing data, hooks to run logic at fixed points in the write lifecycle, and transactions to make several operations succeed or fail together. Later chapters cover the SQL builder for the cases the fluent API does not reach, scopes and sessions for keeping query logic reusable, and the type-safe generics API alongside the traditional one.

The closing chapters are about production: performance, observability, security, and testing against a